29,280,000 token

彻底搞懂

一个开源编程 agent

OpenHands：这一讲怎么走

历史
SDK
单还是多
托管和界面
自动化
问答

修 bug · 编程 agent

retry.py

```
def retry_count(config):
    return config.get("retries") or 3
```

四个测试（输入 → 期望输出）

输入	缺失	0	None	2
期望	3	0	3	2



修 bug 的 agent

要修的 bug

用 SDK 一块一块搭起来

模型和工具 | 对话 | 自定义工具 | 持久化 | 压缩 | Docker | 动作确认

示意



课件 p.1



Simple Frontend Demo 课件 p.2–3

• 历史
SDK
单还是多
托管和界面
自动化
问答

到 2023 年：主流是代码补全

Copilot · 早期的 Cursor

HumanEval 一题

HumanEval

```
def has_close_elements(numbers, threshold):
    """给一串数和一个阈值:
    有没有两个数相差小于阈值"""
    for i, a in enumerate(numbers):
        for b in numbers[i + 1:]:
            if abs(a - b) < threshold:
                return True
    return False
```

示意

输入

一个函数说明

输出

模型写出的实现

判对错 ✓ ✓ ✓

单元测试

改 跑 查 都还是人在主导

设计 ← 教训

2023 补全 | 2024 agent 出现 | 2024 底 管用 | 2025 变强 | 2026 部署



课件 p.4–5

历史
SDK
单还是多
托管和界面
自动化
问答

SWE-bench：拿真实 issue 来考

中文重绘 · 按原图读数

真实 GitHub issue

梯度提升在 warm start 下有数据泄露

仓库快照

- sklearn/
- examples/
- README.rst
- reqs.txt
- setup.cfg
- setup.py

→ **语言模型** →

生成的补丁

- sklearn/
  - gradient_boosting.py
  - helper.py
- utils/

→

修前	修后	仓库自己的测试
✗	✓	join_struct_col
✗	✓	vstack_struct_col
✗	✓	dstack_struct_col
✓	✓	matrix_transform
✓	✓	euclidean_diff

该修好的要过　　原来通过的不能弄坏

最好 4.8%　给了理想检索时（就算把该看的文件直接递给它）

出处：SWE-bench 论文摘要

2023 年 10 月 · SWE-bench

2023 补全 | 2024 agent 出现 | 2024 底 管用 | 2025 变强 | 2026 部署



课件 p.6（中文重绘）

• 历史
SDK
单还是多
托管和界面
自动化
问答

2024 年 3 月：Devin 和 OpenDevin

3 月 12 日 — 3 月 13 日 — 3 月 26 日

Devin 发布演示

演示视频（缩略）

号称第一个 AI 软件工程师

打进要做的事 → 先列计划 → 再动手做

不开源

第二天：网上的号召

“我们来复现 Devin，一起做。”

响应的人非常多

课上判断：这正是大家真想要的东西

OpenDevin 早期前端

前端 | agent | Docker 沙箱



课上举的经历：一天晚上让 Claude 3 照着 Devin 的截图做的

社区复现号召里起步的独立开源项目 · 后来改名 OpenHands

不是 Devin 的官方开源版

2023 补全 | 2024 agent 出现 | 2024 底 管用 | 2025 变强 | 2026 部署



课件 p.8（中文重绘）

• 历史
SDK
单还是多
托管和界面
自动化
问答

学术界也在做：早期的编程 agent

2024 年 4 月

SWE-agent

做 SWE-bench 的那批人发布
第一个开源的这类 agent

搜文件 | 改代码 | 验证修复

借了打开、搜索、
编辑文件这些工具

2024 年 5 月

CodeAct 1.0

OpenDevin 发布

执行代码 | 终端命令 | 拿回反馈

SWE-bench Lite 21% 精简版 · 300 题

agent 和计算机之间的**接口**，成了研究题目

2023 补全 | **2024 agent 出现** | 2024 底 管用 | 2025 变强 | 2026 部署



课件 p.9（中文重绘）

基准分数：模型 + agent 系统

更好的模型

推理 | 改代码 | 用工具

+

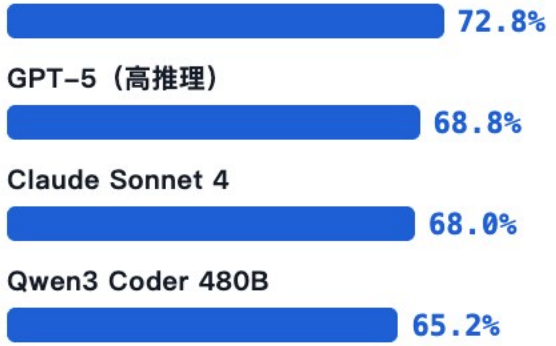
更好的 agent 系统

工具 | 测试环境 | 上下文 | 出错后恢复

↑ **基准分数** ↑

同一套 agent、换不同模型

SWE-Bench Verified



OpenHands SDK 报告表 2

2023 补全 | 2024 agent 出现 | 2024 底 管用 | **2025 变强** | 2026 部署



课件 p.10

• 历史
SDK
单还是多
托管和界面
自动化
问答

2026：主要是部署和界面之年

模型当然还在变好

更长的运行 · 中途被打断 · 并行任务

进度看得见 · 结果能检查

可重复的工作流 · 限定范围的权限 · 验证

历史 → 设计

2023 补全 | 2024 agent 出现 | 2024 底 管用 | 2025 变强 | 2026 部署



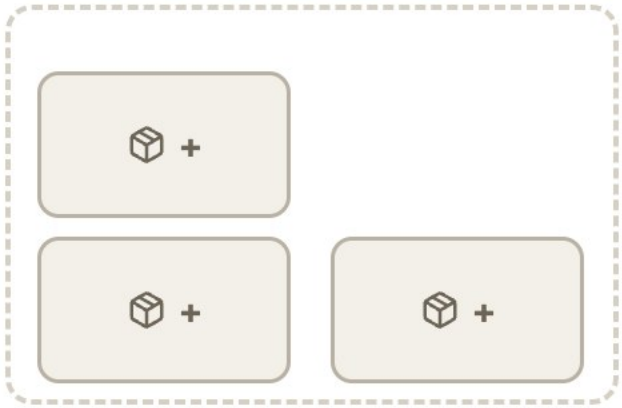
课件 p.11

历史
SDK
单还是多
托管和界面
自动化
问答

和 Pi 正好相反

Pi

先什么都不给，再往上加



OpenHands

默认全有，哪块都能换





课件 p.12—13

第一块：模型和工具

历史
SDK
单还是多
托管和界面
自动化
问答

01_hello_world.py

```python
llm = LLM(model=..., api_key=...)
agent = Agent(
    llm=llm,
    tools=[
        Tool(name=TerminalTool.name),
        Tool(name=FileEditorTool.name),
        Tool(name=TaskTrackerTool.name),
    ],
)
```

= "terminal"，只是名字

列进去的只是工具的名字（规格）
带执行逻辑的 ToolDefinition，由 SDK 按名字去找

选模型

终端 / 编辑器 / 任务清单

终端 → 跑测试

编辑器 → 改那一行代码

任务清单 → 复现、改、跑测试

可复用的 agent 配置



示意



课件 p.14

历史
SDK
单还是多
托管和界面
自动化
问答

第二块：对话 Conversation

工作区
文件在哪 · 命令在哪跑

Conversation
任务 · 动作 · 观察都记在这

run
动作–观察循环

后面讲隔离

hello_world.py

```
conversation = Conversation(agent=agent, workspace=cwd)
conversation.send_message("retry_count 设成 0 时返回了 3，修好它并跑通测试")
conversation.run()
```

任务	动作	观察	动作	观察	动作	观察
修好 retry_count	grep retry_count	找到 2 处	pytest	0 那格失败	改一行	全部通过

示意



课件 p.15

历史
SDK
单还是多
托管和界面
自动化
问答

代码驱动，还是命令行驱动

命令行驱动

发一条命令 · 等它跑完

- ✓ 简单
- ✗ 能传进去的东西有限

代码驱动

在代码里直接调用 · import 进来

- ✓ 哪块都能改
- → 自己实现一个新模块

看你用哪种语言、要改多深

OpenHands：Python

OpenCode、Pi：TypeScript



课堂口头补充

第三块：自定义工具

Action
动作：有类型的输入
`pattern` `path`

→

Executor
执行器：真正干活的实现



→

Observation
观察：模型读得懂的结果
`… retry_count …`
`… retry_count …`

执行器接收动作 → 返回观察

02_custom_tools.py

```python
class GrepAction(Action):
    pattern: str = Field(
        description="Regex to search for")
    path: str = Field(default=".")

class GrepObservation(Observation):
    matches: list[str] = Field(
        default_factory=list)
```

示意

动作
grep
retry_count

观察
找到 2 处

先搜在哪出现 → 再决定改哪

自定义工具和 MCP 工具共用这一套接口
按 SDK 的报告



课件 p.16

第四块：会话持久化

历史
SDK
单还是多
托管和界面
自动化
问答

任务 修好 retry_count → 动作 grep retry_count → 观察 找到 2 处 → 动作 pytest → 观察 0 那格失败 → 动作 改一行 → 观察 全部通过

示意

存进去

同一个 ID
再建一次对话

同一个 ID 读回来

.conversations/<ID>
事件
对话状态

10_persistence.py

```python
conversation = Conversation(
    agent=agent, workspace=cwd,
    persistence_dir="./.conversations",
    conversation_id=conversation_id,
)
```

工作区：另外管

改过的文件还在不在，看工作区怎么管



课件 p.17

第五块：上下文压缩

历史
SDK
单还是多
托管和界面
自动化
问答

压缩用的模型

发给模型看的上下文

开头 + 摘要 + 最近

任务	动作	摘要：跑了几十轮测试，0 那格一直失败	动作	观察	动作
修好 retry_count	grep		读代码	retry.py	改代码

完整事件日志：一条都没删

示意

压缩器示例

```
condenser = LLMSummarizingCondenser(
    llm=llm.model_copy(update={"usage_id": "condenser"}),
    max_size=10, keep_first=2,
)
agent = Agent(llm=llm, tools=tools, condenser=condenser)
```

示例里：超过 10 个事件就压，开头 2 个永远保留

真用时：按轮数 · 按 token 数 · 想更省就设小一点

细节可能丢 → 文件和测试还在

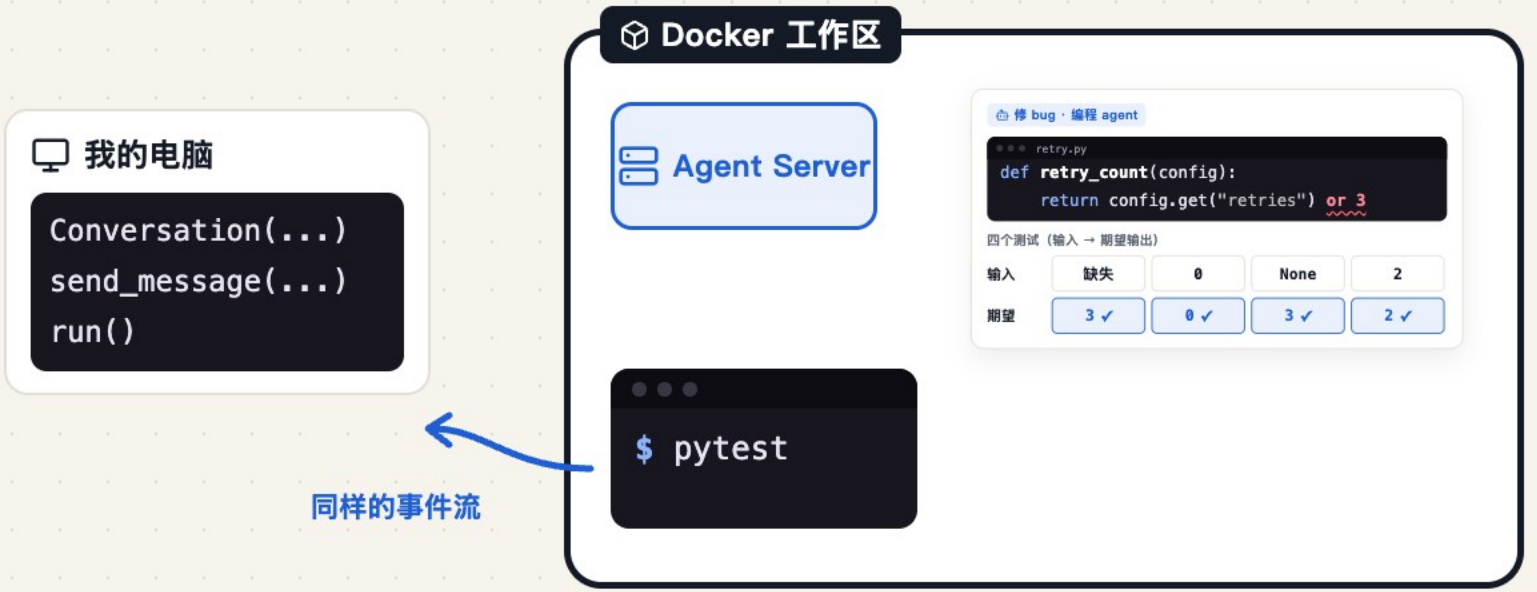




课件 p.18

第六块：远程执行，放进 Docker

历史
SDK
单还是多
托管和界面
自动化
问答



示意

第 13 讲：
边界画在 agent 控制不到的地方

先准备好沙箱，再把 agent 放进去跑

哪种顺手用哪种

用 with 语句，里面的命令都进沙箱



课件 p.19

第七块：动作确认

历史
SDK
单还是多
托管和界面
自动化
问答



总是问 | 从不问（默认） | 按风险问

动作确认示例

```
conversation.set_confirmation_policy(AlwaysConfirm())
```

人来决定

同意 | 拒绝

先开 PR，不要直接推主分支

任务	动作	观察	动作	观察	动作
修好 retry_count	改一行	全部通过	git push	被拒绝	开 PR

示意

风险等级 低 | 中 | 高 | 未知

安全分析器：给每一步打风险等级

ConfirmRisky 默认阈值 HIGH，判断不了的也停下来问

第 13 讲：拦截要在执行之前



课件 p.20

历史
SDK
单还是多
托管和界面
自动化
问答

搭好了：一个修 bug 的 agent

修 bug · 编程 agent

retry.py

```
def retry_count(config):
    return config.get("retries") or 3
```

四个测试（输入 → 期望输出）

输入	缺失	0	None	2
期望	3	0	3	2

示意



修 bug 的 agent


模型和工具


对话


自定义工具


持久化


压缩


Docker


动作确认

搭一个编程 agent 的骨架：七块，照着一块块对


课件 p.12–20

历史
SDK
单还是多
托管和界面
自动化
问答

课堂问答

问 2 模型是不是对 OpenHands 过拟合了?

训练 很多模型训练时用过它，有些没用过

工具 当初几乎照着 Claude 那套来做 → 和 Claude 配合得很好

现在 大多数模型开箱就挺好用



课堂问答

历史
SDK
单还是多
托管和界面
自动化
问答

早期很流行：给 agent 分专门角色

CodeR · 相关：MetaGPT



🔁 每次解决一个问题，都走这一套流程



课件 p.21–22

历史
SDK
单还是多
托管和界面
自动化
问答

单 agent 基线

固定的多 agent 流程



2024 年博客：别小看单 agent 系统



只在目的明确时，才加 agent 至少在编程上

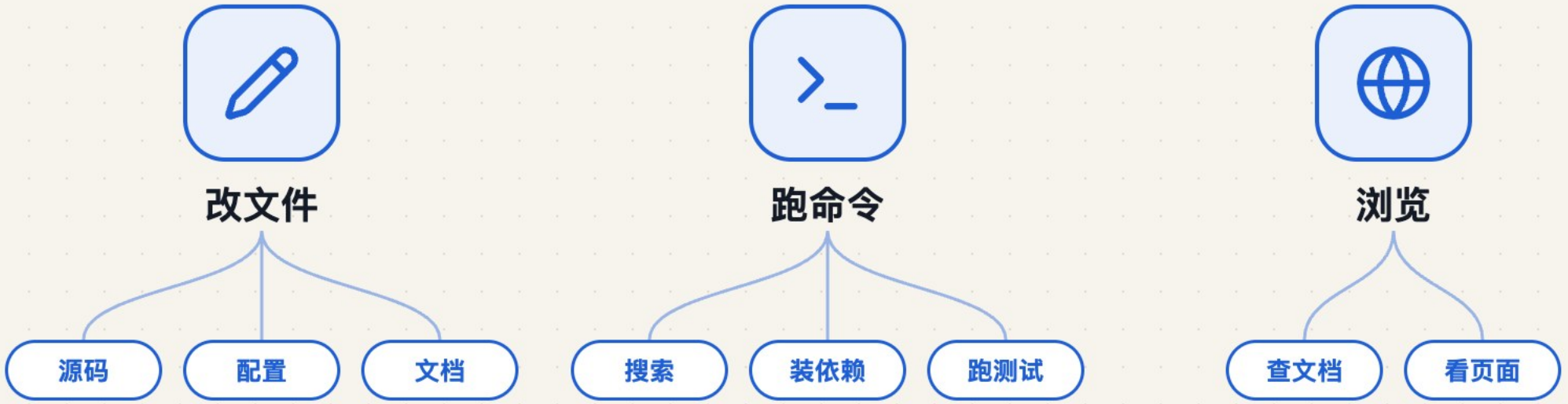
客服这类绝不能乱来的场景：也许还需要固定流程



课件 p.23

历史
SDK
• 单还是多
托管和界面
自动化
问答

一小套工具，组合出很多用法





课件 p.24

历史
SDK
• 单还是多
托管和界面
自动化
问答

技能：需要时才拿进来的提示词

早先叫 microagent · 后来 Anthropic 叫 skill



修 bug 的 agent

改完 push 上去

关键词：docker

用 Docker 要……

示意

agent 的上下文

repo.md

跑测试用 pytest

改完跑全量测试

关键词：push

不直接推主分支，先开 PR

Anthropic： agent 自己挑要读哪个

OpenHands： 按关键词触发

SDK 现在两种都支持 在对的时候，把对的上下文拿进来

skill.py（节选）

```
AgentContext(skills=[Skill(
    name="repo.md",
    content=...,
    trigger=None)])
```



课件 p.25

历史
SDK
单还是多
托管和界面
自动化
问答

MCP 工具：外部服务器提供

多 agent：每个角色背着自己的

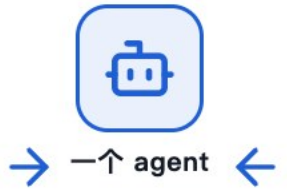
角色 提示词 工具

角色 提示词 工具

角色 提示词 工具



一个 agent + 技能 + MCP



每个角色的提示词和工具：需要的那一刻再拿进来

一个 agent 也能做完所有的事



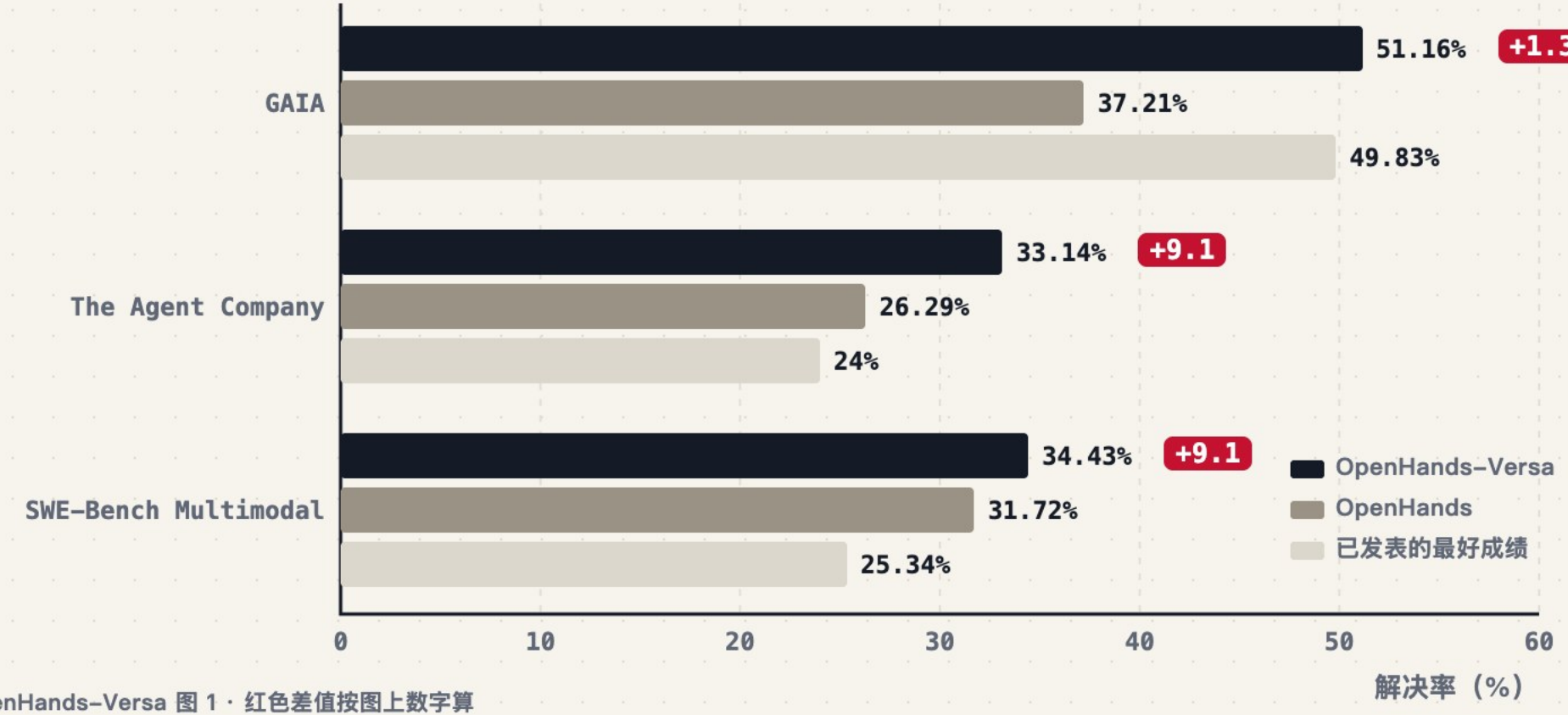
课件 p.26

历史
SDK
单还是多
托管和界面
自动化
问答

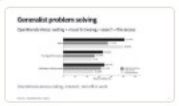
同一套通用工具，超过更专门的 agent

OpenHands-Versa：写代码 + 能看图的浏览 + 搜索 + 读文件

一段时间以前的工作：绝对分数不高



出处：OpenHands-Versa 图 1 · 红色差值按图上数字算



课件 p.27 图 1

历史
SDK
单还是多
托管和界面
自动化
问答

多 agent 什么时候值得分工

并行

十个 PR 同时跑

彼此独立的 issue 或仓库，大多能拆开



示意

独立审查

去掉上下文的 agent 来审

据课上的经验

审自己的活	不带上下文
对自己客气	下手狠得多

0 和没设置还是没分清

控制成本

子 agent 回报时：扔掉用过的上下文，只带回要紧的

- 合适的任务：便宜的模型
- 聪明的模型：负责统筹

代价：任务边界 · 共享上下文 · 协调成本

带回恰好够用的上下文 · 不冲突 · 不重复实现 · 整合回仓库

深度调研

分出去容易得多（第 10 讲）



课件 p.28

历史
SDK
单还是多
托管和界面
自动化
问答

本地还是云：放在哪里跑

本地电脑

好处 现成的文件、服务和上下文

要求 执行期间机器得一直开着

人在环里：修 bug、做功能

云端工作区

好处 合上笔记本也能接着干

要求 远端的仓库、依赖、凭证

后台：代码审查、非编程

Devin
一开始在云端

Claude Code · Codex
直接在本地跑

大家更爱本地 课上说

第 13 讲讲隔离技术，这里讲用户怎么选



课件 p.29–30

历史
SDK
单还是多
托管和界面
自动化
问答

四种界面，都能配本地或云

执行位置 × 交互界面

	本地	云	例子
IDE	编辑器工作区	远程工作区	VS Code 这类编辑器
图形界面 GUI	桌面应用	浏览器应用	开头演示的那个前端
命令行 CLI	本地终端	远程会话	Claude Code
集成	本地服务	托管的工作进程	Slack、GitHub



课件 p.31（中文重绘）

历史
SDK
单还是多
托管和界面
自动化
问答

自定义图形界面：围绕 agent 任务



进度和并行的运行

预览、diff 和批准

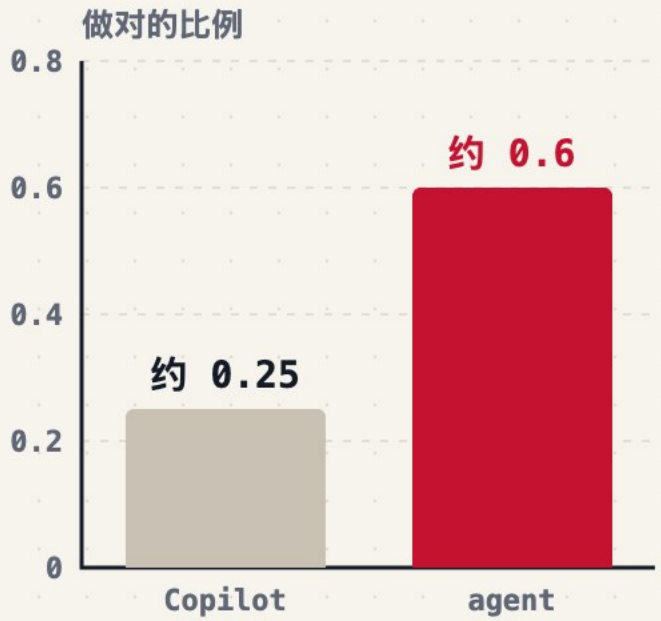
工作区设置

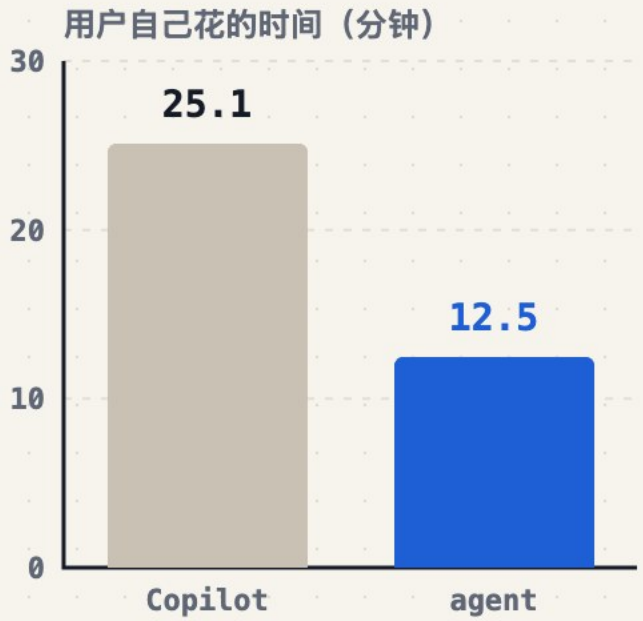


课件 p.33（中文重绘）

历史
SDK
单还是多
托管和界面
自动化
问答

主动投入少了，但仍要理解和监督





20 名常用 Copilot 的学生 · Python 任务 · 每种工具 40 分钟

Code with Me or for Me? 图 4 · 正确率按原图读数

看它干活好无聊 课上说

↓

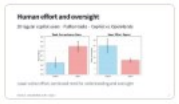
帮人同时做别的事

我没法确认它做对没有

论文里：难控制，要理解的东西太多

↓

帮人验证输出



课件 p.34（中文重绘）

历史
SDK
单还是多
• 托管和界面
自动化
问答

命令行和集成：就在你本来的地方



服务器 >_

家里的电脑 >_

看图要另开预览



示意

不用换界面

课上认为：这会越来越普遍



课件 p.36（中文重绘）

历史
SDK
单还是多
托管和界面
自动化
问答

自动化面板：一个个机器人

PR 审查
事件

多仓库 PR 审查
定时

issue 分诊
事件

修失败的依赖升级
定时

清理过期 CI
定时

Slack 频道监控
被 @

触发方式：事件 · 定时 · 被 @ 成功 失败

示意



课件 p.37–38

事件 → 有边界的任务 → 能检查的结果

历史
SDK
单还是多
托管和界面
自动化
问答

维护者看一眼

触发	上下文	运行	产物	检查
issue · PR · 定时	仓库 · 指令 · 权限	隔离的工作区	补丁 · 报告 · 评论	人或规则

拿例子走一遍：retry_count 的 issue

issue：retry_count 设成 0 时返回了 3

有人提了 issue

这个仓库

分诊指令

只读权限



在一个容器里

验收标准
① 设成 0 时不重试
② 没设置时才用默认的 3
③ 原有测试全部通过

一条评论：验收标准

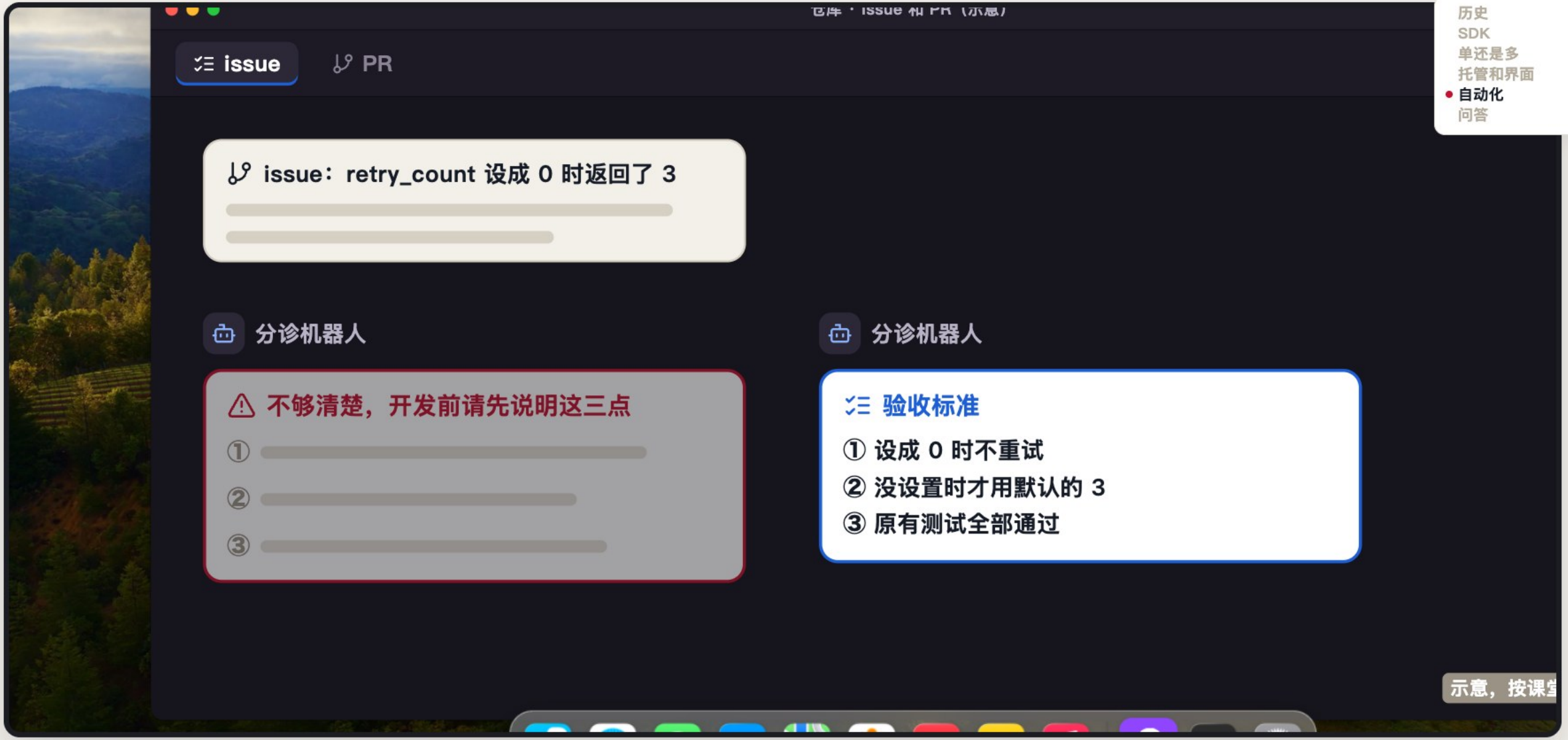
维护者看一眼

第 13 讲：最小权限 · 沙箱

示意

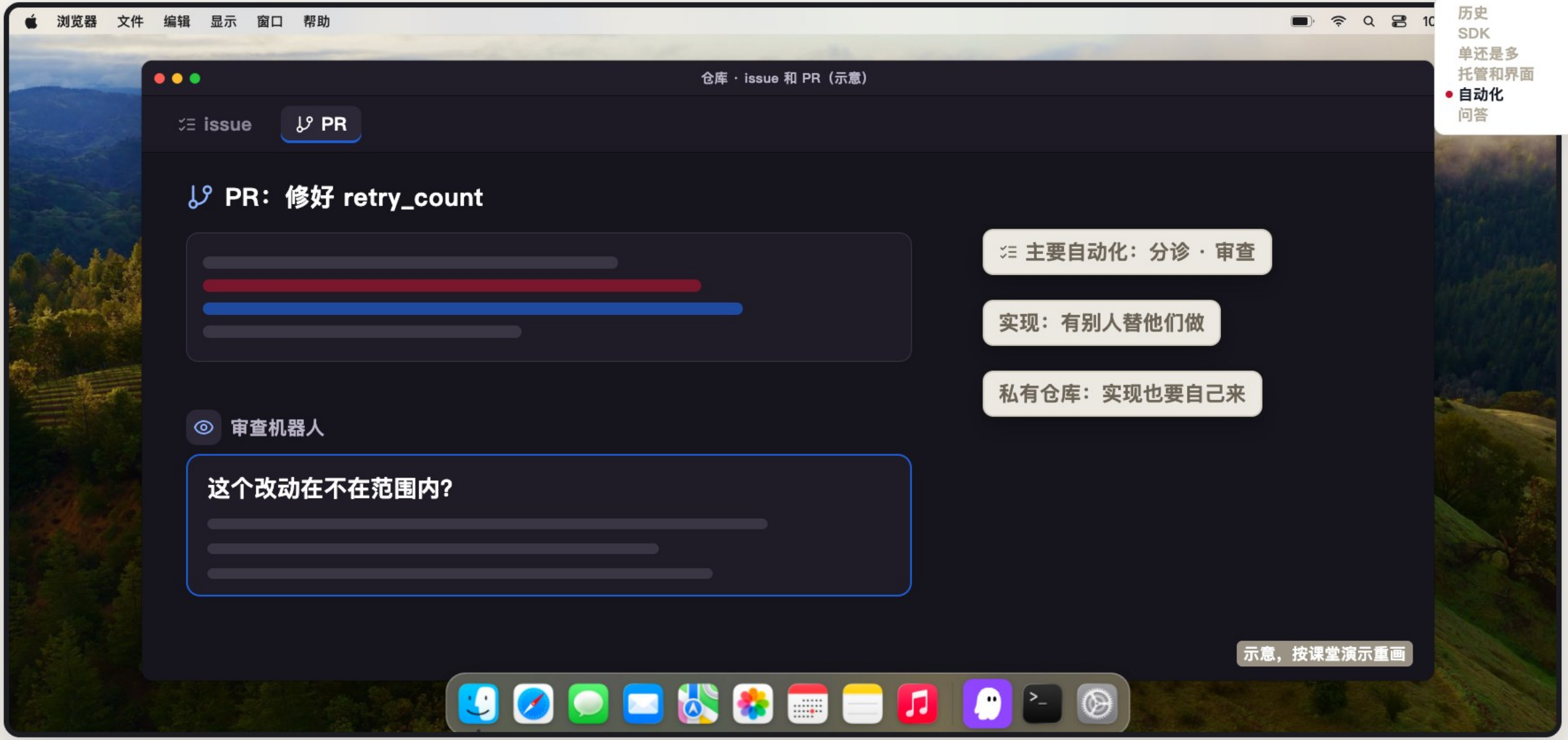


课件 p.39


issue
PR
issue：retry_count 设成 0 时返回了 3
分诊机器人
不够清楚，开发前请先说明这三点
①
②
③
分诊机器人
验收标准
① 设成 0 时不重试
② 没设置时才用默认的 3
③ 原有测试全部通过
历史
SDK
单还是多
托管和界面
自动化
问答
示意，按课堂



课件 p.40



历史
SDK
单还是多
托管和界面
自动化
问答



课件 p.40

软件工厂：反复出现的任务 · 明确的交接 · 检查

历史
SDK
单还是多
托管和界面
自动化
问答

issue 分诊
分诊写出验收标准

→

实现
交上补丁和测试

→

代码审查
不带上下文的 agent

→

合并检查
等批准
✓ 合进去

交接单
需求和验收标准

交接单
补丁 · 测试 · PR

交接单
独立验证

当前提交上测试全过 + 有人批准



示意



课件 p.41

从过去的运行里学：SkillRefiner

从审查和后续修改里拿到**延迟的反馈**，变成对技能的修改

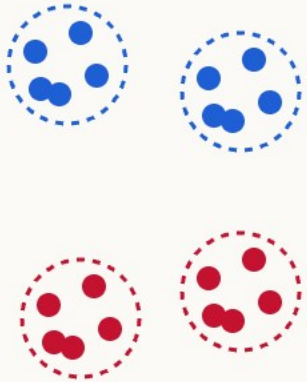
历史
SDK
单还是多
托管和界面
• 自动化
问答

① 摘要分类 — ② 向量聚类 — ③ 提修改 — ④ 证据把关 — ⑤ 合并技能（合并）

① 摘要分类 — 每条轨迹写摘要

- 成功 / 摘要
- 失败 + 证据（证据） / 摘要
- 机器人：看着不错 →
  维护者：不行，要改

② 向量聚类 — 成功、失败分开聚类



③ 提修改 — 每个簇提一条修改

- 正面提议
- 负面提议

④ 证据把关 — 只看负面提议

- 负面提议



证据把关

通过 ✓

⑤ 合并技能 — 合进原来的技能

- 原来的技能
- + 提议
- 新技能

离线 · 只用历史记录 · 不重新跑



课件 p.42

历史
SDK
单还是多
托管和界面
自动化
问答

每个设置都比初始技能好

留出任务上的准确率（%）· PR 审查是 F1 · 数字按原图读数

任务	初始技能	模型	初始	GEPA	Trace2Skill	SkillRefiner	
表格任务 SpreadsheetBench	模型写的	GPT-5.4-mini	77.5	78.5	79.0	84.0 ↑	
	模型写的	MiniMax-M2.7	68.0	75.0	69.5	76.0 ↑	
	人写的	GPT-5.4-mini	86.0	86.0	86.5	86.5 ↑	打平
	人写的	MiniMax-M2.7	79.0	73.0	81.5	84.5 ↑	
数学 DAPO-Math	模型写的	GPT-5.4-mini	90.0	90.0	91.0	91.0 ↑	打平
	模型写的	MiniMax-M2.7	53.5	53.5	61.0	64.0 ↑	
PR 审查（F1）	人写的	GPT-5.4-mini	0.16	–	0.13	0.18 ↑	
	人写的	MiniMax-M2.7	0.11	–	0.13	0.15 ↑	

精炼这一步花的 token：比对比的方法少 1.4–42×

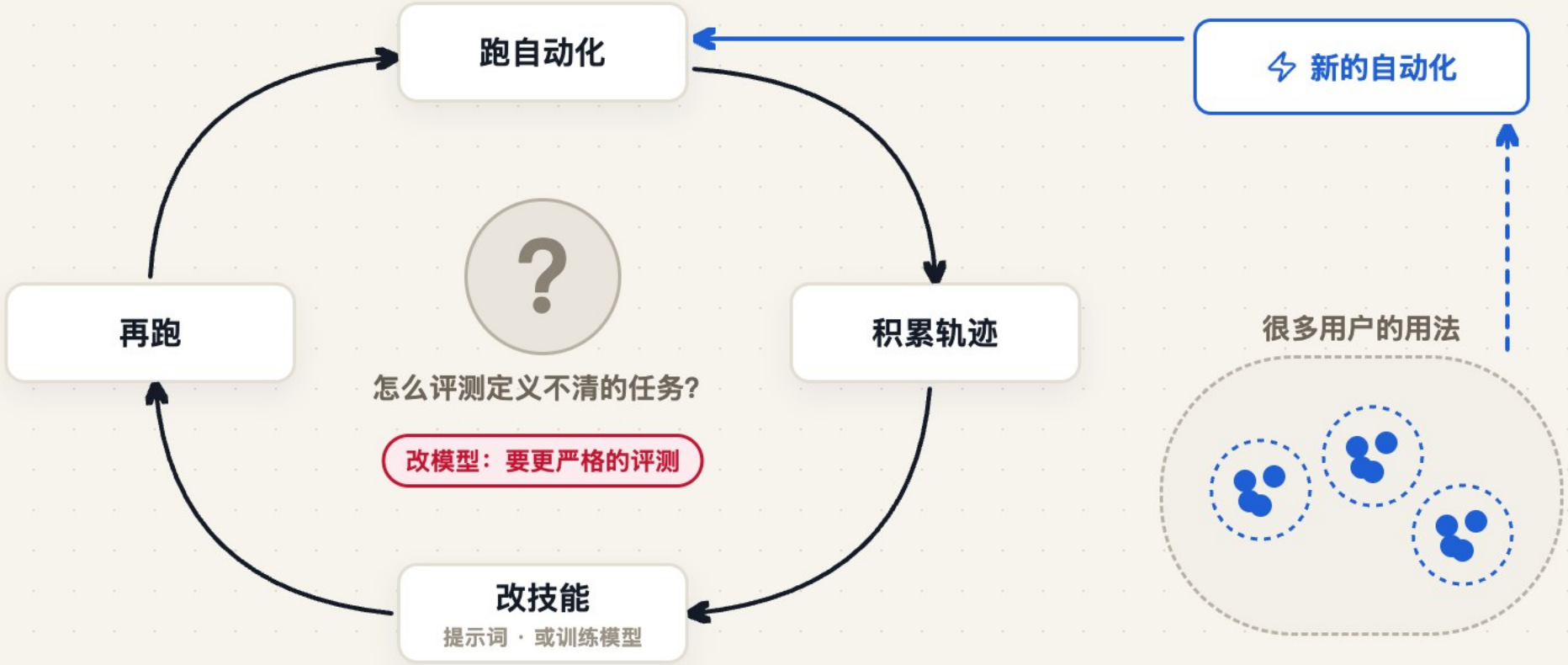
一：GEPA 要重新跑任务，PR 审查的反馈要等几天到几周，没法重放

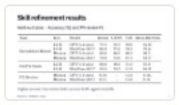


课件 p.43 · 论文表 1

往后：越用越好

历史
SDK
单还是多
托管和界面
自动化
问答





课堂口头补充

历史
SDK
单还是多
托管和界面
自动化
问答

问 2 多台机器上的 agent，怎么同步个人上下文?



一 都存在 GitHub 或者云上，每次重新拉

二 常用服务都接上 MCP 集成，不绑定在某一处



课后问答

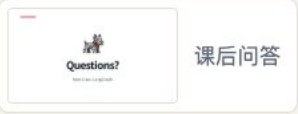
历史
SDK
单还是多
托管和界面
自动化
问答

问 3 能 SSH 进所有机器的 agent，放在哪?

先小心 agent 一旦被攻破，所有机器跟着失守

更好 开一条隧道，让 agent 住在那台机器上





课后问答

历史
SDK
单还是多
托管和界面
自动化
问答

问 4 多 agent 是在省 token 吗?

课上说 不一定：前面那项工作里，多 agent 更贵，但成功率高得多

原因 它们把活做完了，没有提前停下

所以 便宜不能当成默认





示意：只画相对高低



课后问答

历史
SDK
单还是多
托管和界面
自动化
问答

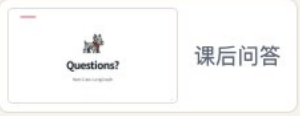
问 5 做 agent 式开发，一定要用 GitHub 吗?

有替代品 比如 GitLab

不该 重写整个 GitHub：它背后有大量工程

该想的 怎么验证工作做得好

真正值得重新想的：怎么验证



课后问答

OpenHands：串起来

01

历史

- 从代码补全走到部署
- 分数来自模型加 agent 系统

代码补全
SWE-bench
OpenDevin
部署

模型 + agent 系统

02

SDK

- 默认什么都配好
- 每一块都能换



七块积木

03

单还是多

- 单 agent 加技能和 MCP
- 分工只在少数时候值得



并行 · 独立审查 · 控成本

04

托管和界面

- 用户更爱本地
- 无聊、没法验证



无聊　没法验证

05

自动化

- 同一件事一遍遍地跑
- 从历史里学习

历史 → 技能 ↺

软件工厂



课件 p.44

放回修 bug 的 agent

模型和工具 | 对话 | 自定义工具 | 持久化 | 压缩 | Docker | 动作确认

我的电脑

```
Conversation(...)
send_message(...)
run()
```

同样的事件流

Docker 工作区

Agent Server

```
$ pytest
```

修 bug · 编程 agent

retry.py

```
def retry_count(config):
    return config.get("retries") or 3
```

四个测试（输入 → 期望输出）

输入	缺失	0	None	2
期望	3 ✓	0 ✓	3 ✓	2 ✓

等确认

`>_ git push`

开 PR

验收标准

① 设成 0 时不重试
② 没设置时才用默认的 3
③ 原有测试全部通过

示意



课件 p.44

CMU 11-768 · AI AGENTS

第 14 讲 · OpenHands

下一讲：第 15 讲 · 编排和监控

LangGraph

每晚 20:00 更新



课件 p.44